

AWS Code Signing

AWS Lambda Code Signing

1) What is Code Signing in AWS Lambda?

Code Signing ensures that:

- Only **trusted, verified, and untampered code** is deployed to Lambda.
- It uses **digital signatures** to validate:
 - Who created the code (authenticity)
 - Whether it was modified (integrity)

👉 *“Lambda code signing enforces that only code signed by trusted entities can be deployed, preventing unauthorized or tampered code execution.”*

 End-to-End Flow (Diagram)



How Signing Works Internally

1. Code is hashed (SHA-based)
2. Hash is encrypted using signer's private key
3. Signature is attached to the package

4. Lambda:

- Decrypts using public key
- Recomputes hash
- Compares both

👉 If mismatch → deployment fails

Without Code Signing ❌

- Anyone with access can upload:
 - Malicious code
 - Backdoor scripts

With Code Signing ✅

- Only code signed via your **AWS Signer profile** is accepted
- Unauthorized upload → blocked

🔒 Security Benefits

- Prevents:
 - Code tampering
 - Unauthorized deployments
 - Supply chain attacks
- Ensures:
 - Code integrity
 - Trusted CI/CD pipelines

⚠️ Limitations

- Requires setup (Signer + IAM)
- Not supported for containers
- Adds extra deployment step



Lambda Under the Hood

Lambda Under the Hood

Lambda internally uses:

- **Firecracker microVMs (lightweight VMs)** on EC2
- Each invocation runs inside an isolated environment

👉 Important:

You don't manage EC2, but Lambda runs on top of it.

Execution Lifecycle

Step-by-step lifecycle:

1. Request comes in

- Via API Gateway, S3, EventBridge, etc.

2. Scheduler decides:

- Is there a **warm container available?**

3A. Warm Start Path 🔥

- Reuse existing environment
- Skip initialization
- Execute immediately

3B. Cold Start Path ❄️

- Create new environment:
 - Allocate compute (EC2-based microVM)
 - Attach runtime (Node, Python, etc.)
 - Download code
 - Initialize dependencies

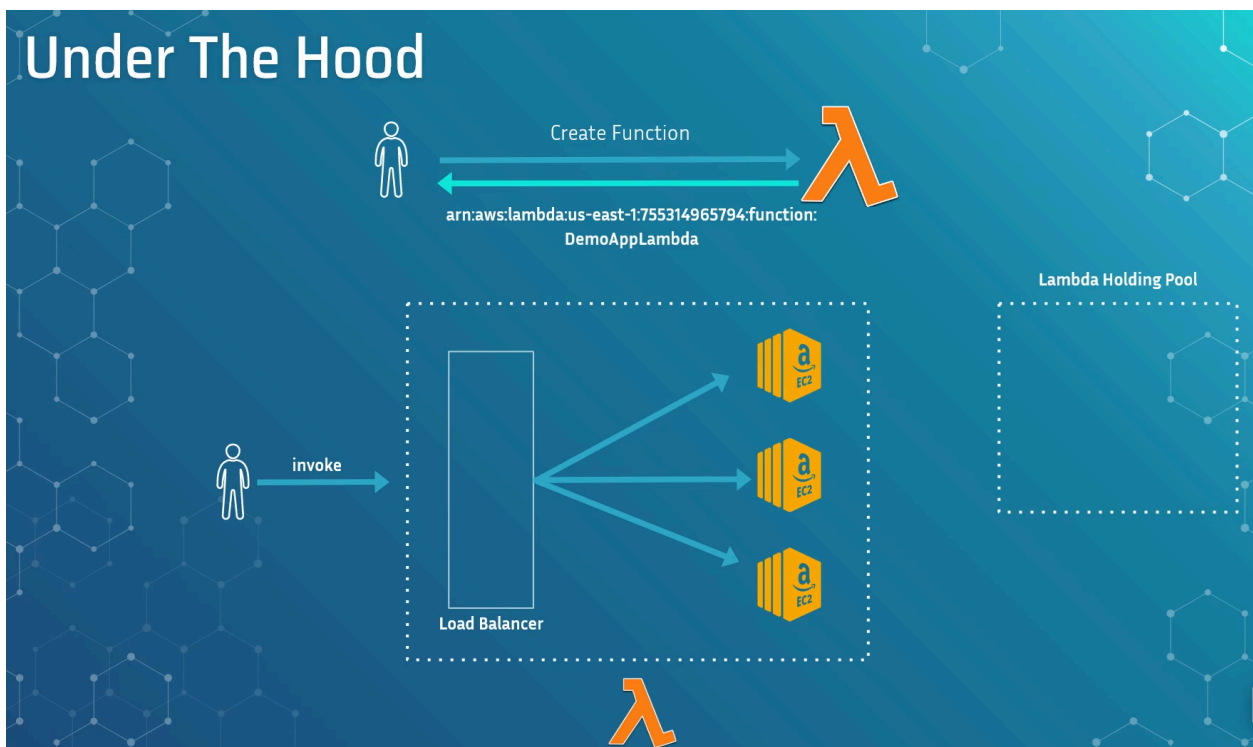
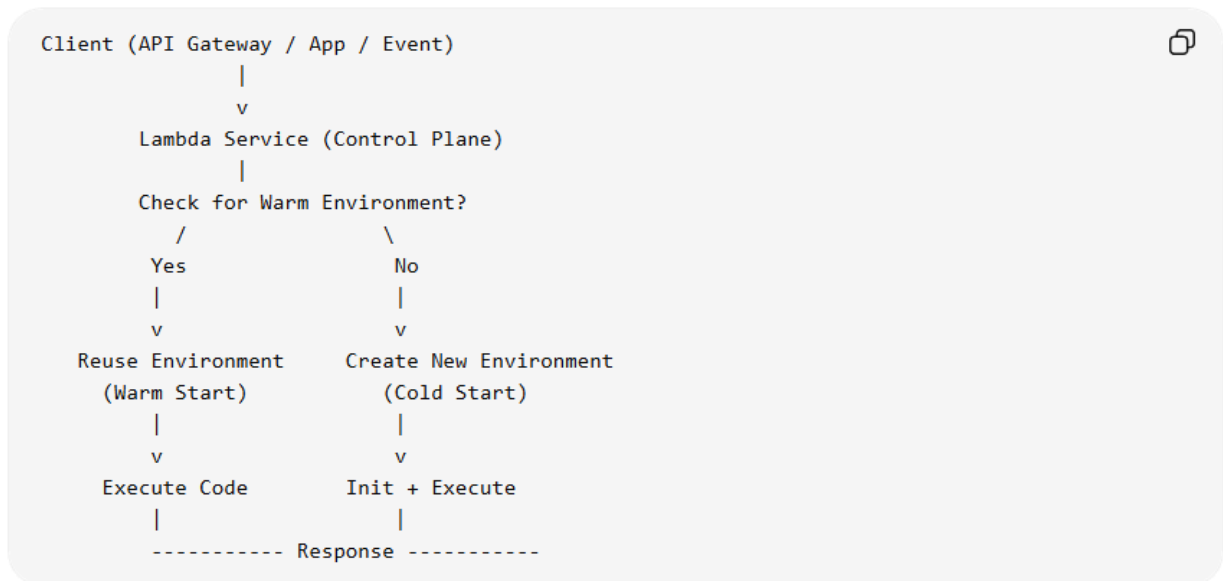
4. Run handler

- Your function executes

5. Return response

6. Environment kept alive (temporary)

- May be reused → “holding pool”



Interview Questions

Q1: What is Lambda invocation?

👉 Execution of function in response to an event.

Q2: What is cold start?

👉 Initialization delay when no warm environment exists.

Q3: What is warm start?

👉 Reuse of existing execution environment.

Q4: Why do cold starts happen?

👉 New environment creation due to scaling or inactivity.

Q5: What affects cold start time?

- Runtime (Java slower than Node)
- Package size
- Dependencies
- VPC config

Q6: What is latency in Lambda?

👉 Total time from request to response.

Q7: Does Lambda use EC2 internally?

👉 Yes, via microVMs (Firecracker), abstracted from user.

Q8: What is execution environment reuse?

👉 Reusing initialized container for multiple requests.

Q9: Can we control warm environments?

👉 Not directly, but via Provisioned Concurrency.

Q10: Difference between cold start and execution time?

Cold Start

Execution

Setup time

Business logic

Happens
sometimes

Always
happens

Q11: API is slow only on first request. Why?

👉 Cold start

Q12: Traffic suddenly spikes. What happens?

👉 New instances → cold starts

Q13: After 10 mins idle, API is slow again. Why?

👉 Warm environments destroyed

Q14: How to reduce cold start?

- Provisioned Concurrency
- Smaller package
- Avoid heavy dependencies
- Use lighter runtime



Trigger in AWS Lambda

Trigger in AWS Lambda

“Means to invoke your function”

👉 A trigger is anything that causes your Lambda function to run.

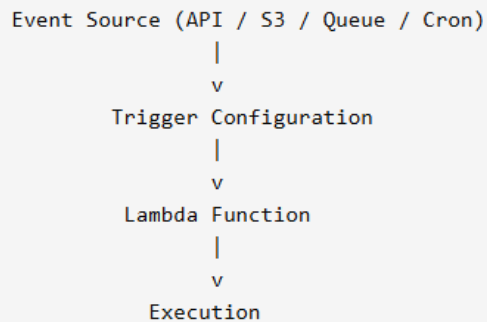
OR

👉 *A trigger is an event source or service that invokes a Lambda function automatically or manually.*

Types of Invocation

1. **Direct Invocation**
 - a. SDK / CLI / Console
2. **Service-based Invocation**
 - a. Other AWS services trigger Lambda

Lambda Trigger Architecture



Common Triggers

- 🕒 1. Amazon CloudWatch Events (Now EventBridge)
 - Scheduled jobs (cron)

Example:

- Run cleanup job every day at 2 AM

Cron → EventBridge → Lambda → Cleanup DB

♦ **Interview Point:**

👉 Used for **time-based automation**

2. Amazon API Gateway

- REST APIs

Example:

Client → API Gateway → Lambda → Response

👉 Your Node.js backend APIs typically use this

♦ **Features:**

- Auth (JWT, Cognito)
- Rate limiting
- Request validation

3. Amazon SNS / Amazon SQS

♦ **SNS (Push Model)**

- Publishes message → triggers Lambda immediately

♦ **SQS (Pull Model via Event Source Mapping)**

- Lambda polls queue

Example:

Order Placed → SNS → Lambda → Send Email

Order Queue (SQS) → Lambda → Process Orders

4. Amazon DynamoDB Streams

- React to DB changes

Example:

User Insert → DynamoDB Stream → Lambda → Send Welcome Email

- ◆ Trigger Type:

👉 Event Source Mapping (not direct push)

5. Amazon S3

- File upload / update

Example: Upload Image → S3 → Lambda → Resize Image

Event Source Mapping

“Similar concept called Event Source Mapping”

- ◆ What is it?

👉 A configuration where Lambda polls data sources

- ◆ Used for:

- SQS
- DynamoDB Streams
- Kinesis

Scaling Behavior

“Different triggers have different scaling / retry behaviour”

Retry Behavior

- ◆ Async Triggers (S3, SNS)

- Retry automatically
- Dead Letter Queue supported

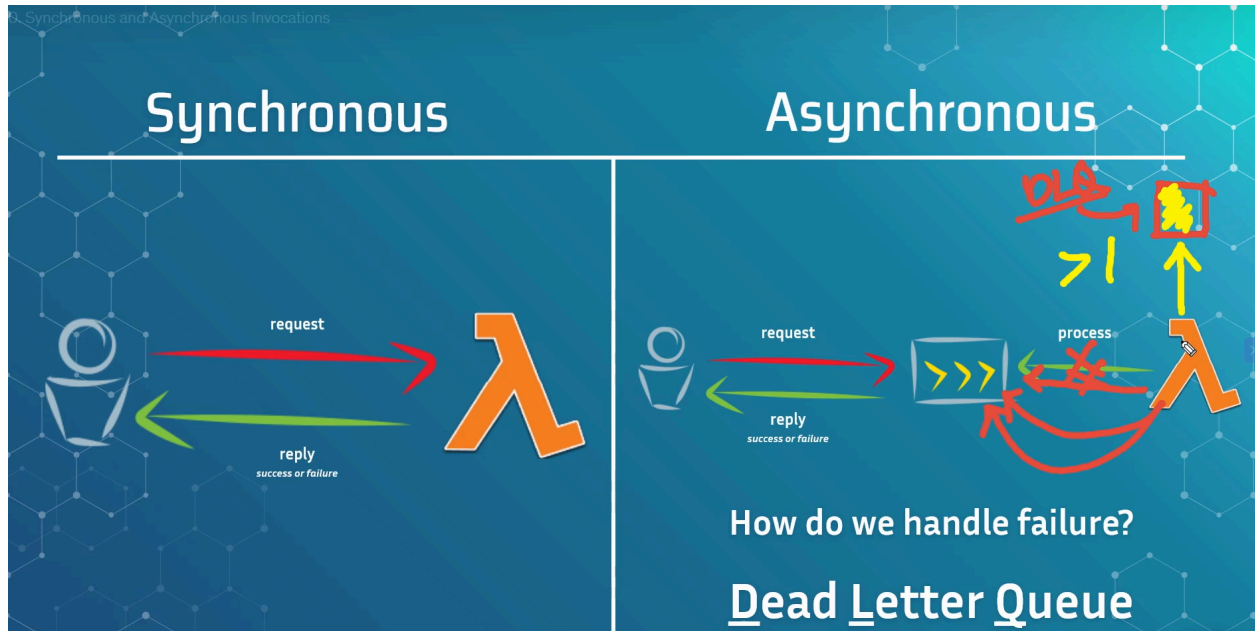
- ◆ SQS

- Retries until message processed
- Uses visibility timeout



Synchronous vs Asynchronous Invocation

🚀 Synchronous vs Asynchronous Invocation



🚀 1. Synchronous vs Asynchronous Invocation in AWS Lambda

Client —request→ Lambda
Client ←response— Lambda (success / failure)

✅ Synchronous Invocation (SYNC)

- ◆ Definition: Client **waits** for Lambda to finish and return response

- ◆ **Flow Explanation**

1. Client sends request
2. Lambda executes
3. Lambda returns result
4. Client continues

- ◆ **Example :** User → API Gateway → Lambda → Response (JSON)

👉 Typical REST API

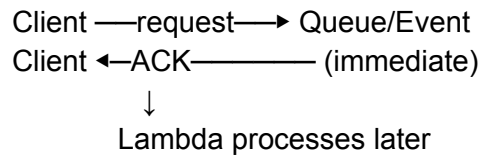
- ◆ **Key Characteristics**

- Blocking (wait required)

- Immediate response
- Errors returned directly
- No automatic retry

♦ Real Example (Your Backend)

GET /users
 → API Gateway
 → Lambda
 → DB query
 → Return response



✓ Asynchronous Invocation (ASYNC)

♦ Definition: Client **does NOT wait**, Lambda processes later

♦ Flow Explanation

1. Client sends request
2. Event stored (queue/event system)
3. Client gets immediate ACK
4. Lambda processes in background

♦ Example

User uploads file → S3 → Lambda → Process later

♦ Key Characteristics

- Non-blocking
- Event-driven
- Automatic retries
- Fault tolerant

Dead Letter Queue (DLQ)

◆ What is DLQ?

👉 A queue where **failed events are stored after retries**

◆ Services Used:

- Amazon SQS
- Amazon SNS

◆ Flow

Lambda fails
↓
Retry (2–3 times)
↓
Still fails
↓
Send to DLQ

◆ Why DLQ?

- Prevent data loss
- Debug failures
- Reprocess later

5. Async Retry Behavior

◆ Default:

- Retries 2 times (can configure)

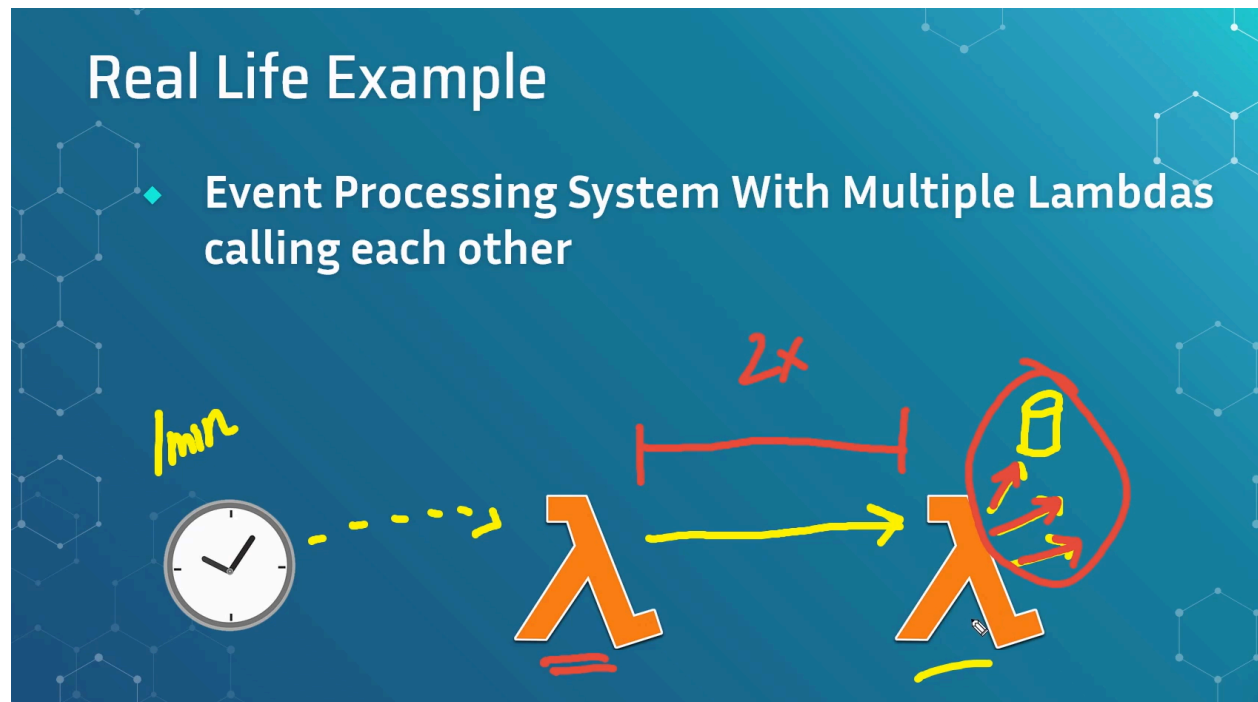
◆ With delay:

- Exponential backoff

◆ If still fails:

- Goes to DLQ

🧠 Real-Life Example



Scenario: Event Processing System

Every 1 min (Cron)

↓
Lambda A (in case of Async cost of Lambda A get increase due to waiting)

↓
Lambda B (async call)

↓
Queue / Retry

↓
DLQ if failed

◆ Explanation

- Lambda A triggers Lambda B
- Lambda B processes data
- If fails:
 - Retry happens
 - If still fails → DLQ

When to Use SYNC vs ASYNC

Use SYNC When:

- You need immediate response
 - APIs (login, fetch data)
 - Client must know result
-

Use ASYNC When:

- Background processing
- File upload
- Notifications
- Heavy workloads



Concurrency & Throttling



Concurrency & Throttling



What is Concurrency?

Concurrency = number of Lambda executions running at the same time

Requests → | R1 | R2 | R3 | R4 |

Lambda → | E1 | E2 | E3 | E4 | (4 concurrent executions)

Each request → one execution environment.

♦ Example

- 100 users hit your API at the same time
👉 Lambda may run **100 concurrent executions**

♦ Formula (useful in interviews)

$\text{Concurrency} \approx \text{Requests per second} \times \text{Average execution time (seconds)}$

Example:

- 50 req/sec
- 2 sec execution

👉 $50 \times 2 = 100$ concurrency needed



2) Account-Level Concurrency Limit

Every AWS account has a limit per region.

♦ Default

- **~1000 concurrent executions per region** (soft limit)
-

♦ Visual

Total Account Concurrency = 1000

Function A → 300

Function B → 200

Function C → 500

Total → 1000 (limit reached)

What is Throttling?

♦ Definition

When Lambda **rejects requests** because concurrency limit is exceeded

♦ Visual

Limit = 1000

Incoming Requests = 1200

1000 → Processed

200 →  Throttled

♦ Example

- Your API suddenly gets traffic spike
👉 Exceeds limit → requests fail
-

Types of Concurrency

♦ A) Unreserved Concurrency

♦ Definition

Default shared pool for all Lambda functions

◆ Important Rule

AWS keeps 100 concurrency reserved automatically

👉 So usable:

1000 total - 100 reserved buffer = 900 usable

◆ Visual

Total Limit = 1000

Reserved for system = 100

Unreserved Pool = 900

◆ Behavior

- All functions share this pool
 - No guarantees
 - One function can consume all
-
-



B) Reserved Concurrency

◆ Definition

Guarantees AND limits concurrency for a specific function

◆ Two Roles

1. Guarantee capacity

2. Limit max concurrency

◆ Visual

Total = 1000

Function A → Reserved = 300

Function B → Reserved = 200

Remaining (Unreserved) = 500

◆ Example

Payment API → Reserved = 200

👉 Even during traffic spike:

- Payment API always gets 200 slots (and if requests get more than 200 then all requests above 200 will fail.)
Get request 250 then it processes 200 requests and fail 50 requests.
-

◆ Special Case

Reserved = 0

👉 Function is **disabled (throttled completely)**

C) Provisioned Concurrency

◆ Definition

Pre-warmed execution environments (NO cold start)

◆ Visual

Provisioned = 5

Ready Environments:

[E1] [E2] [E3] [E4] [E5] (always warm)

◆ Behavior

- Always ready
 - No cold start latency
 - Used for low-latency APIs
-

◆ Example

Login API → Provisioned = 10

👉 Always instant response

🔥 Reserved vs Provisioned (Important)

Feature	Reserved	Provisioned
Guarantees capacity	✓	✓
Prevents cold start	✗	✓
Limits max concurrency	✓	✗
Cost	Free	Paid

⚠️ Throttling Scenarios

◆ Scenario 1: Account Limit Exceeded

Total = 1000
Incoming = 1200
→ 200 throttled

◆ Scenario 2: Reserved Limit Hit

Function limit = 100
Incoming = 150
→ 50 throttled

◆ Scenario 3: Burst Traffic

Lambda scales gradually:

- Not instant infinite scaling

Interview Questions (Complete Set)

◆ Basic

Q1: What is concurrency?

👉 Number of parallel Lambda executions

Q2: What is throttling?

👉 Rejecting requests due to limit exceeded

◆ Intermediate

Q3: Default concurrency limit?

👉 ~1000 per region

Q4: What is unreserved concurrency?

👉 Shared pool for all functions

Q5: Why reserve 100 concurrency?

👉 AWS keeps buffer for critical operations

◆ Advanced

Q6: Difference between reserved and provisioned?

👉 Reserved = limit + guarantee

👉 Provisioned = pre-warmed instances

Q7: Can a function use both?

👉 ☒ Yes

Q8: What happens if reserved > account limit?

👉 Not allowed

◆ Scenario-Based

Q9: One function consuming all concurrency?

👉 Use **reserved** concurrency

Q10: Cold start problem?

👉 Use **provisioned** concurrency

Q11: API getting throttled?

👉 Increase limit OR optimize concurrency



Version, Alias & Enviroment



Version, Alias & Environment



1) Lambda Version

◆ Definition

A version is an immutable snapshot of your Lambda function code + configuration.

OR

Versions are the snapshots of the code and settings of a function.

◆ What gets stored in a version?

- Code (ZIP / container reference)
- Runtime (Node, Python, etc.)
- Memory, timeout
- Environment variables
- Layers

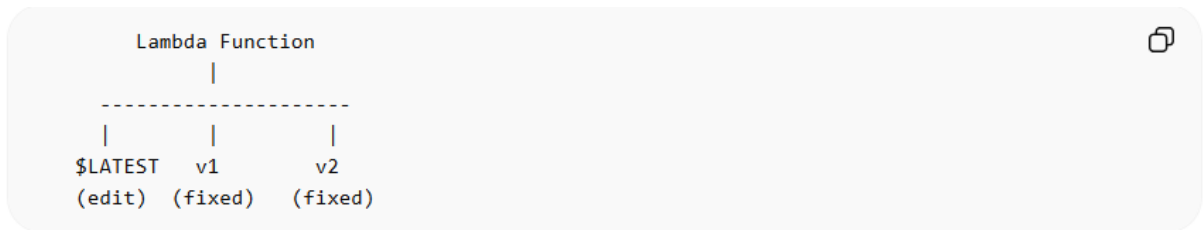
👉 Once published → **cannot be changed**

◆ Default Version

\$LATEST

- Mutable version
 - Updated every time you deploy
-

◆ Visual



◆ Example

aws lambda publish-version

👉 Creates:

- v1, v2, v3...

◆ Key Rule (Interview ★)

Versions are **read-only snapshots**

◆ Limitations

- Each version consumes **storage (75 GB total limit per region)**
- Cannot modify after creation
- Too many versions → storage issues

🎯 2) Lambda Alias

◆ Definition

An alias is a named pointer to a specific version

◆ Why alias?

👉 Instead of calling version directly (v1, v2), you use:

- prod
- dev
- beta

◆ Visual

Versions:

v1 v2 v3

Aliases:

prod → v2

dev → v3

test → v1

◆ Example

```
aws lambda create-alias --name prod --function-version 2
```

◆ Advanced Feature: Traffic Shifting

prod alias:

90% → v2

10% → v3

👉 Used for:

- Canary deployment
 - Blue-green deployment
 - Suppose you have redirected all traffic to Prod alias the when you deploy new version then you link prod to that new version so you don't have to lambda version mapping anywhere else.
-

◆ Limitations

- Alias points to **only one primary version**
 - Weighted routing supports only **2 versions**
 - Cannot point to **\$LATEST**
-
-

3) Lambda Environment Variables

◆ Definition

Key-value pairs used to configure Lambda at runtime

◆ Example

```
{  
  "DB_URL": "mongodb://...",  
  "API_KEY": "xyz123"  
}
```

◆ Usage (Node.js)

```
const db = process.env.DB_URL;
```

◆ Visual

```
Lambda Function  
  |  
  v  
Environment Variables  
(DB_URL, API_KEY, MODE)
```

◆ Important Behavior

- Stored per version
 - Immutable after version publish
 - Can be different for each version
-
-

🔥 4) Version + Alias + Environment Together

◆ Combined Flow

```
Alias (prod)
  |
  v
Version v2
  |
  v
Env Variables (PROD DB)
```

◆ Example

Version	Env	Use
v1	Dev DB	testing
v2	Prod DB	live

Alias:

prod → v2
dev → v1

! 5) Default Values

◆ Version

- Default: `$LATEST`
-

◆ Alias

- No default alias
 - Must create manually
-

◆ Environment Variables

- Default: none
- Max size: **4 KB total**

7) Limits & Constraints

◆ Versions

- Max depends on storage (75 GB total)
 - Each version = full copy
-

◆ Aliases

- Limited per function (soft limit ~50)
-

◆ Environment Variables

- Max size: 4 KB
 - Key/value strings only
 - Encrypted using KMS
-
-

6) Common Interview Mistakes

✗ Alias = copy of version
👉 Wrong → it's just a pointer

✗ Versions can be edited
👉 Wrong → immutable

✗ Env variables are global
👉 Wrong → version-specific

8) Interview Questions

◆ Basic

Q1: What is Lambda version?

👉 Immutable snapshot of function

Q2: What is alias?

👉 Pointer to version

Q3: What is \$LATEST?

👉 Mutable working version

◆ Intermediate

Q4: Why use alias?

👉 Easy deployment + version abstraction

Q5: Can alias point to multiple versions?

👉 Yes (weighted routing)

Q6: Are env variables version-specific?

👉 ✅ Yes

◆ Advanced

Q7: How to do zero-downtime deployment?

👉 Use alias + traffic shifting

Q8: Can we modify a published version?

👉 ❌ No

Q9: How to manage multiple environments?

👉 Use:

- Versions
 - Aliases
 - Env variables
-
-

◆ Scenario-Based

Q10: You want dev & prod setup?

dev → v1 → Dev DB
prod → v2 → Prod DB

Q11: Rollback deployment?

👉 Point alias back to old version

DLQ , Layers & Urls

DLQ , Layers & Function Urls

1) Dead Letter Queue (DLQ)

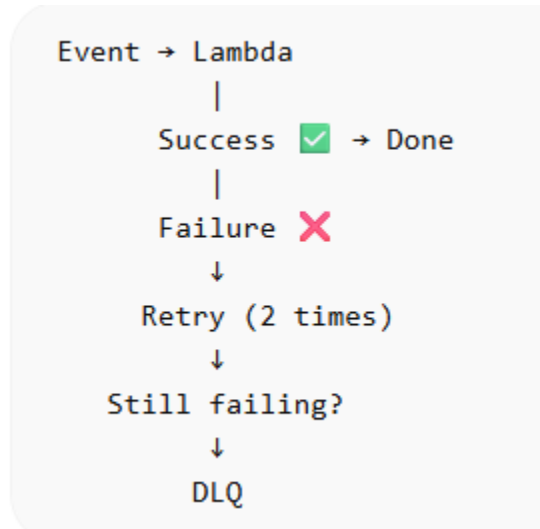
◆ Definition

A DLQ is a destination where failed events are stored after retries are exhausted.

◆ Supported services

- Amazon SQS
 - Amazon SNS
-

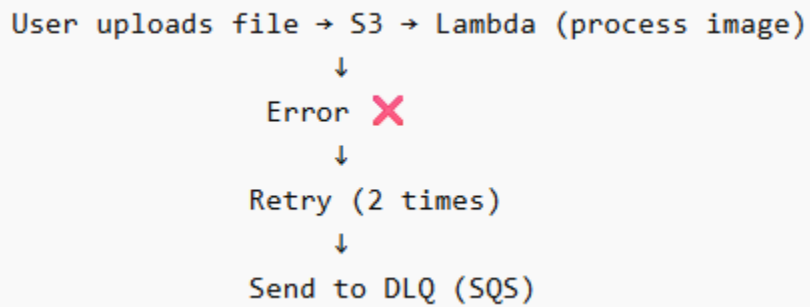
Flow (from async invocation)



◆ When DLQ is used

- Async invocations (S3, SNS, EventBridge)
 - Not for sync (API Gateway)
-

♦ Example



♦ Why DLQ?

- Prevent data loss
- Debug failures
- Reprocess later

♦ Default Behavior

- Retries: 2 times
- No DLQ unless configured

♦ Limitations

- Only for async invocation
- Stores event, not execution logs
- Requires manual reprocessing

♦ DLQ vs Destinations (Advanced)

Feature	DLQ	Destination
Stores failed events	✓	✓
Stores success events	✗	✓
Flexible routing	✗	✓

2) Lambda Layers

◆ Definition

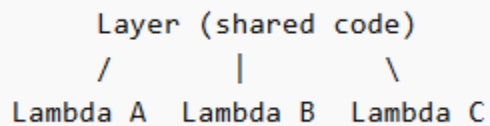
A Layer is a shared package of code/dependencies used by multiple Lambda functions.

◆ Why use Layers?

👉 Avoid duplicating:

- node_modules
 - Python packages
 - Common utilities
-

◆ Visual



◆ Example

Instead of:

Lambda A → includes express

Lambda B → includes express

Use:

Layer → express

Lambda A → uses layer

Lambda B → uses layer

♦ Real Example

Layer:

- mongoose
- axios

Used by:

- User API
 - Order API
-

♦ Key Behavior

- Layers are mounted at runtime
 - Read-only
 - Versioned
-

♦ Limits

- Max 5 layers per function
 - Max size:
 - 50 MB (zipped)
 - 250 MB (unzipped total with function)
-

♦ Default

- No layers attached
-

♦ Benefits

- Smaller deployment package
- Faster updates
- Reusability

♦ Drawbacks

- Cold start can increase slightly
- Version management needed

3) Lambda Function URL

◆ Definition

A Function URL is a built-in HTTPS endpoint to invoke Lambda directly (without API Gateway)

◆ Visual

Client → HTTPS URL → Lambda → Response

◆ Example

`https://abc123.lambda-url.aws.com/`

◆ Use Case

- Simple APIs
 - Webhooks
 - Internal tools
-

◆ Auth Types

Type	Description
NONE	Public access
AWS_IAM	Secured via IAM

◆ Example

POST /lambda-url
→ Lambda executes
→ Returns JSON

4) Interview Questions

◆ **Basic**

Q1: What is DLQ?

👉 Queue for failed events

Q2: What is Lambda Layer?

👉 Shared dependency package

Q3: What is Function URL?

👉 Direct HTTPS endpoint for Lambda

◆ **Intermediate**

Q4: When is DLQ triggered?

👉 After retries fail

Q5: Max layers per function?

👉 5

Q6: Difference between API Gateway and Function URL?

👉 API Gateway = full features

👉 Function URL = simple endpoint

◆ **Advanced**

Q7: DLQ vs Destinations?

👉 Destinations more flexible

Q8: Can layers be shared across functions?

👉 ☒ Yes

Q9: Are layers versioned?

👉 ☒ Yes

◆ **Scenario-Based**

Q10: Large dependencies shared across APIs?

👉 Use Layers

Q11: Want simple webhook endpoint?

👉 Use Function URL

Q12: Want to debug failed async events?

👉 Use DLQ



Limitation



Lambda Limitation

Code | Test | Monitor | **Configuration** | Aliases | Versions

Configuration

- General configuration**
- Triggers
- Permissions
- Destinations
- Function URL
- Environment variables

General configuration [Info](#) [Edit](#)

Description	Memory	Ephemeral storage
-	128 MB	512 MB
Timeout	SnapStart	Info
0 min 3 sec	None	

Lambda Duration Limit

1. Maximum 15 minutes before automatic timeout.
2. This means Lambda are not suitable for long running processes.
3. Lambdas will automatically Timeout when the configured limit reached.
4. Default is 3 second but we can configure it to 15 minutes.

Lambda Memory Limit

1. Defines the amount of memory (RAM) available to your function at runtime.
2. “Lambda memory is not just RAM, it controls CPU and performance.”
3. Range between 128MB and 10240MB(10 GB).
4. Changing Memory is the most significant performance lever available to you.
5. Increasing memory determines instance type behind the scenes.
6. Increasing memory setting can reduce the impact of cold start.

Version Limitations

1. Each version consumes **storage (75 GB total limit per region)**
2. Cannot modify after creation
3. Too many versions → storage issues

♦ Alias Limitations

- Alias points to **only one primary version**
- Weighted routing supports only **2 versions**
- Cannot point to **\$LATEST**

Environment Variables Limitation

- Max size: **4 KB total**
- Key/value strings only
- Encrypted using KMS

DLQ Limitations

- Only for async invocation
- Stores event, not execution logs
- Requires manual reprocessing

Layer Limitation

- Max 5 layers per function
- **Max size: (same as lambda function)**
 - **50 MB (zipped)**
 - **250 MB (unzipped total with function)**

Lambda upload Limitation

- **50 MB (zipped)**
- **250 MB (unzipped total with function)**